REACT 状态逻辑复用

— 林宜丙

状态逻辑 VS 逻辑状态

状态逻辑的复用方式

- Mixins
- HOC
- Render props

```jsx
export default class extends React.Component {
  constructor(props) {
    super(props);
    this.state = {
      width: window.innerWidth,
      height: window.innerHeight
    };
  }
  componentDidMount() {
    window.addEventListener("resize", this.handleResize);
  }
  componentWillUnmount() {
    window.removeEventListener("resize", this.handleResize);
  }
  handleResize = () => {
    this.setState({
      width: window.innerWidth,
      height: window.innerHeight
    });
  }
  render() {
    let {
      width,
      height
    } = this.state;

    return (
      <div>
        <div>窗口宽度: {width}</div>
        <div>窗口高度: {height}</div>
      </div>
    );
  }
}
```


窗口宽度: 676
窗口高度: 958

mixins.js

```
const WindowSize = {
  getInitialState () {
    return {
      width: window.innerWidth,
      height: window.innerHeight
    };
  },
  handleResize() {
    this.setState({
      width: window.innerWidth,
      height: window.innerHeight
    });
  },
  componentDidMount() {
    window.addEventListener("resize", this.handleResize);
  },
  componentWillUnmount() {
    window.removeEventListener("resize", this.handleResize);
  }
};

export default React.createClass({
  mixins: [WindowSize],
  render() {
    let { width, height } = this.state;

    return (
      <div>
        <div>窗口宽度: {width}</div>
        <div>窗口高度: {height}</div>
      </div>
    );
  }
})
```

319 ▲ 0 LF UTF-8 JavaScript Not on branch GitHub Git (20)

缺点

- 引入隐含的依赖
- 命名空间冲突
- 复杂度滚雪球般增大

HOC.js

```
const withWindowSize = ComponentToWrap => {
  return class WindowSizeHOC extends React.Component {
    constructor(props) {
      super(props);
      this.state = {
        width: window.innerWidth,
        height: window.innerHeight
      };
    }
    componentDidMount() {
      window.addEventListener("resize", this.handleResize);
    }
    componentWillUnmount() {
      window.removeEventListener("resize", this.handleResize);
    }
    handleResize = () => {
      this.setState({
        width: window.innerWidth,
        height: window.innerHeight
      });
    }
    render() {
      <ComponentToWrap {...this.state}/>
    }
  }
}

class WindowSizeRender extends React.Component {
  render() {
    let { width, height } = this.props;

    return (
      <div>
        <div>窗口宽度: {width}</div>
        <div>窗口高度: {height}</div>
      </div>
    );
  }
}
export default withWindowSize(WindowSizeRender);
```

872 ▲ 0 + src/WindowSize/HOC.js 29:1 LF UTF-8 JavaScript Browser

缺点

- 难以溯源
- 覆盖问题
- 静态构建
- 增加无用层级

render-props.js

```
class WindowSize extends React.Component {
  constructor(props) {
    super(props);
    this.state = {
      width: window.innerWidth,
      height: window.innerHeight
    };
  }
  componentDidMount() {
    window.addEventListener("resize", this.handleResize);
  }
  componentWillUnmount() {
    window.removeEventListener("resize", this.handleResize);
  }
  handleResize = () => {
    this.setState({
      width: window.innerWidth,
      height: window.innerHeight
    });
  }
  render() {
    this.props.children(this.state);
  }
}
export default class WindowSizeRender extends React.Component {
  render() {
    let { width, height } = this.props;
    return (
      <WindowSize>
        {({ width, height }) => (
          <div>
            <div>窗口宽度: {width}</div>
            <div>窗口高度: {height}</div>
          </div>
        )}
      </WindowSize>
    );
  }
}
```

872 ▲ 0 src/WindowSize/render-props.js LF UTF-8 JavaScript Browser

缺点

- 组件嵌套地狱
- 使用嵌套表达组合
- 无法使用 SCU 优化

“

这可如何是好?

They let you use state and other React features without writing a class.

-React document

hooks.js

```
function useWindowSize() {
  let [ width, setWidth ] = useState(window.innerWidth);
  let [ height, setHeight ] = useState(window.innerHeight);

  React.useEffect(() => {
    let handleResize = () => {
      setWidth(window.innerWidth);
      setHeight(window.innerHeight);
    };
    window.addEventListener("resize", handleResize);
    return () => {
      window.removeEventListener("resize", handleResize);
    };
  });

  return {
    width,
    height
  };
}

export default function () {
  const {
    width,
    height
  } = useWindowSize();

  return (
    <div>
      <div>窗口宽度: {width}</div>
      <div>窗口高度: {height}</div>
    </div>
  )
}
```

Hooks

319 ▲ 0 LF UTF-8 JavaScript Not on branch GitHub Git (20)

钩点

动机

- 组件状态逻辑难以复用
- 复杂组件变得难以理解
- 类组件让人和机器都感到迷惑

复杂组件难以理解

ClassApp.js

```jsx
import * as React from "react";  8.5kB (gzipped: 3.4kB)
import { Form, Input } from "antd";  670.2kB (gzipped: 201.2kB)
import { ThemeContext } from "./ThemeContext";

export default class extends React.Component {
  constructor(props) {
    super(props);
    this.state = {
      name: "Hary",
      surname: "Potter",
      width: window.innerWidth,
      height: window.innerHeight
    };
  }
  componentDidMount() {
    window.addEventListener("resize", this.handleResize);
    document.title = this.state.name + " " + this.state.surname;
  }
  componentDidUpdate() {
    document.title = this.state.name + " " + this.state.surname;
  }
  componentWillUnmount() {
    window.removeEventListener("resize", this.handleResize);
  }
  handleNameChange = ({ target }) => {
    this.setState({ name: target.value });
  }
  handleSurnameChange = ({ target }) => {
    this.setState({ surname: target.value });
  }
  handleResize = () => {
    this.setState({ width: window.innerWidth, height: window.innerHeight });
  }
  render() {
    let {
      name,
      surname,
      width,
      height
    } = this.state;
    return (
      <ThemeContext.Consumer>
        {theme => (
          <Form style={theme.pink}>
            <Form.Item label="Name">
              <Input value={name} onChange={this.handleNameChange} />
            </Form.Item>
            <Form.Item label="SurName">
              <Input value={surname} onChange={this.handleSurnameChange} />
            </Form.Item>
            <div style={{ marginTop: "20px" }}>
              <div>窗口宽度: {width}</div>
              <div>窗口高度: {height}</div>
            </div>
          </Form>
        )}
      </ThemeContext.Consumer>
    );
  }
}
```

hooks.js

FunctionApp.js

```jsx
import React, { useContext, useState, useEffect } from "react";
import { Form, Input } from "antd";
import { ThemeContext } from "./ThemeContext";

export default function(props) {
  let theme = useContext(ThemeContext);

  let [name, setName] = useState("Hary");
  let [surname, setSurname] = useState("Potter");
  useEffect(() => {
    document.title = name + " " + surname;
  });

  let [width, setWidth] = useState(window.innerWidth);
  let [height, setHeight] = useState(window.innerHeight);
  useEffect(() => {
    let handleResize = () => {
      setWidth(window.innerWidth);
      setHeight(window.innerHeight);
    };
    window.addEventListener("resize", handleResize);
    return () => {
      window.removeEventListener("resize", handleResize);
    };
  });

  return (
    <Form style={theme.white}>
      <Form.Item label="Name">
        <Input value={name} onChange={({ target }) => setName(target.value)} />
      </Form.Item>
      <Form.Item label="SurName">
        <Input
          value={surname}
          onChange={({ target }) => setSurname(target.value)}
        />
      </Form.Item>
      <div style={{ marginTop: "20px" }}>
        <div>窗口宽度: {width}</div>
        <div>窗口高度: {height}</div>
      </div>
    </Form>
  );
}
```

319 ▲ 0 src/ClassApp.js 63:1 Inline Git Diff Lines: 61 Prettier LF UTF-8 JavaScript Not on branch GitHub Git (23) Browser

类组件让人迷惑

类组件让人和机器都感到迷惑

函数组件
VS
类组件

一个简单的函数组件

function.js ● class.js

```
1  import React from "react";  22.2kB (gzipped: 7.5kB)
2
3  function ProfilePage(props) {
4    const showMessage = () => {
5      alert("您已成功关注" + props.user);
6    };
7
8    const handleClick = () => {
9      setTimeout(showMessage, 5000);
10   };
11
12   return <button onClick={handleClick}>关注他</button>;
13 }
14
15 export default ProfilePage;
16
17
```

0 0 + × ~/Downloads/functi • LF UTF-8 JavaScript (JSX) GitHub Git (0) 14:26

转写为类组件

function.js | class.js

```
import React from "react";  22.2kB (gzipped: 7.5kB)

class ProfilePage extends React.Component {
  showMessage = () => {
    alert("您已成功关注" + this.props.user);
  };

  handleClick = () => {
    setTimeout(this.showMessage, 3000);
  };

  render() {
    return <button onClick={this.handleClick}>关注他</button>;
  }
}

export default ProfilePage;
```

0 0 ~/Downloads/class.j LF UTF-8 JavaScript (JSX) GitHub Git (0) 14:28

index.js

```jsx

import ProfilePageFunction from "./ProfilePageFunction";
import ProfilePageClass from "./ProfilePageClass";

export default class App extends React.Component {
  state = {
    user: "林宜丙"
  };
  render() {
    return (
      <>
        <label>
          <b>请选择用户查看简介: </b>
          <select
            value={this.state.user}
            onChange={e => this.setState({ user: e.target.value
          >
            <option value="林宜丙">林宜丙</option>
            <option value="崔天泽">崔天泽</option>
            <option value="朱俊蓓">朱俊蓓</option>
          </select>
        </label>
        <h1>{this.state.user}的个人简介</h1>
        <p>
          <ProfilePageFunction user={this.state.user} />
          <b> (function component)</b>
        </p>
        <p>
          <ProfilePageClass user={this.state.user} />
          <b> (class component)</b>
        </p>
      </>
    );
  }
}
```

Git

Atom Browser

http://localhost:3001/

请选择用户查看简介: 林宜丙

林宜丙的个人简介

关注他 **(function component)**

关注他 **(class component)**

示例

319 ▲ 0 src/Weibo/index.js 45:1 Inline Git Diff Lines: 37 Prettier LF UTF-8 JavaScript (JSX) Not on branch GitHub Git (23) Browser

不可变数据

解决方法

ProfilePageClass.js

```
import React from "react";

class ProfilePage extends React.Component {
  showMessage = () => {
    alert("您已成功关注" + this.props.user);
  };

  handleClick = () => {
    setTimeout(this.showMessage, 3000);
  };

  render() {
    return <button onClick={this.handleClick}>关注他</button>;
  }
}

export default ProfilePage;
```

ProfilePageClassImmutable.js

```
import React from "react";  8.5kB (gzipped: 3.4kB)

class ProfilePage extends React.Component {
  showMessage = (user) => {
    alert("您已成功关注" + user);
  };

  handleClick = () => {
    const { user } = this.props;

    setTimeout(() => this.showMessage(user), 3000);
  };

  render() {
    return <button onClick={this.handleClick}>关注他</button>;
  }
}

export default ProfilePage;
```

一个简单的函数组件

function.js ● class.js

```
1  import React from "react";  22.2kB (gzipped: 7.5kB)
2
3  function ProfilePage(props) {
4    const showMessage = () => {
5      alert("您已成功关注" + props.user);
6    };
7
8    const handleClick = () => {
9      setTimeout(showMessage, 5000);
10   };
11
12   return <button onClick={handleClick}>关注他</button>;
13 }
14
15 export default ProfilePage;
16
17
```

0 0 + × ~/Downloads/functi • LF UTF-8 JavaScript (JSX) GitHub Git (0) 14:26

类组件让机器迷惑

机器压缩组件代码（*Minify*）

function.js | class.js | untitled

```
var Person = function () {
  function Person(_ref) {
    var name = _ref.name, age = _ref.age, sex = _ref.sex;
    _classCallCheck(this, Person);

    this.className = 'Person';
    this.name = name;
    this.age = age;
    this.sex = sex;
  }

  _createClass(Person, [{ // 副作用
    key: 'getName',
    value: function getName() {
      return this.name;
    }
  }]);
  return Person;
}();
```

0 0 untitled* 12:33 LF UTF-8 JavaScript GitHub Git (0) 15:25

提前编译组件代码（*AOT*）

```
// foo.js
module.exports = function Foo(props) {
  if (props.data.type === 'img') {
    return <img src={props.data.src} className={props.className} alt={props.alt} />;
  }
  return <span>{props.data.type}</span>;
}
Foo.defaultProps = {
  alt: "An image of Foo."
};

// css-class.js
var CSSClasses = {
  bar: 'bar'
};
module.exports = CSSClasses;

// bar.js
var Foo = require('Foo');
var Classes = require('Classes');
module.exports = function Bar(props) {
  return <Foo data={{ type: 'img', src: props.src }} className={Classes.bar} />;
}
```

当 *CSSClasses & defaultProps* 为不可变数据时

```
1  var Foo = require('Foo');
2  var Classes = require('Classes');
3  function Bar(props) {
4    return <Foo data={{ type: 'img', src: props.src }} className={Classes.bar} />;
5  }
6  function Bar_optimized(props) {
7    return <img src={props.src} className="Bar" alt="An image of Foo." />;
8  }
```

```
1
2  function Bar_optimized(props) {
3    return <img src={props.src} className="Bar" alt="An image of Foo." />;
4  }
5
```

优点

- 组件状态逻辑复用方便，UI 和状态逻辑分离更彻底
- 从生命周期转到 Hooks，代码更紧凑不破碎
- 更少的心智负担，利于机器优化

谢谢~